A Project Report on

Malicious URL Detection Using Machine Learning Techniques

Submitted To

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, ANANTAPUR

In partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Submitted by

IV B. Tech II Semester

CHENCHI REDDY GARI NAVYA	**(21KH1A3305)**
SHAIK SAMREEN SULTANA	**(21KH1A3350)**
MARELLA SRAVAN KUMAR	**(21KH1A3315)**
GAJJALA DIVAKAR REDDY	**(21KH1A3311)**

Under the Esteemed Guidance of

Dr.B.LAKSHMI NARAYANA REDDY, M.Tech.,Ph.D
HOD & Professor CSE (AI & ML)

Department of CSE (Artificial Intelligence and Machine Learning)

SRI VENKATESWARA COLLEGE OF ENGINEERING
(Affiliated to JNTUA, Anantapuramu and Approved by AICTE, New Delhi)
Balaji Nagar, Kadapa-516003.

(2021-2025)

Malicious URL Detection Using Machine Learning Techniques

Project Report Submitted in Partial Fulfillment of The Requirement for The Award of the Degree of ***B. Tech in CSE (AI&ML)***

By

CHENCHI REDDY GARI NAVYA	**21KH1A3305**
SHAIK SAMREEN SULTANA	**22KH5A3350**
MARELLA SRAVAN KUMAR	**21KH1A3315**
GAJJALA DIVAKAR REDDY	**21HK1A3311**

Under the Esteemed Guidance of

Dr.B.LAKSHMI NARAYANA REDDY, M.Tech.,Ph.D
HOD & Professor CSE (AI & ML)

Department of CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

SRI VENKATESWARA COLLEGE OF ENGINEERING
(Affiliated to JNTUA, Anantapuramu and approved by AICTE, New Delhi)
Balaji Nagar, Kadapa – 516003.

(2021-2025)

SRI VENKATESWARA COLLEGE OF ENGINEERING

(Affiliated to JNTUA, Anantapuramu and Approved by AICTE, New Delhi)
Balaji Nagar, Kadapa-516003.

Department of CSE (AI&ML)

Certificate

This is to certify that the project work entitled

Malicious URL Detection Using Machine Learning Techniques

is the bonafide work done by

CHENCHI REDDY GARI NAVYA	21KH1A3305
SHAIK SAMREEN SULTANA	22KH5A3350
MARELLA SRAVAN KUMAR	21KH1A3315
GAJJALA DIVAKAR REDDY	21KH1A3311

In the Department of CSE (AI&ML), Sri Venkateswara College of Engineering, Kadapa is affiliated to JNTUA-Anantapur in partialfulfillment of the requirements for the award of Bachelor of Technology in CSE(AI&ML) during 2021-2025.

This work has been carried out under my guidance and supervision.

The results embodied in this Project report have not been submitted in any University or Organization for the award of any degree or diploma.

GUIDE
Dr.B.Lakshmi Narayana Reddy M. Tech., Ph.D
HoD & Professor CSE(AI&ML)

HEAD OF THE DEPARTMENT
Dr.B.Lakshmi Narayana Reddy M . Tech.,Ph.D

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We are extremely thankful to our beloved chairman and establisher **Sri A. Gangi Reddy** for providing necessary infrastructure and resources for the accomplishment of our project at Sri Venkateswara College of engineering, Kadapa.

We are highly indebted to **Dr. R. Veera Sudarsana Reddy, Principal** of **Sri Venkateswara College of Engineering, Kadapa**, for his support during the tenure of the project.

We are very much obliged to our beloved **Dr. B. Lakshmi Narayana Reddy, Head of the Department of CSE (AI & ML)**, Sri Venkateswara College of Engineering for providing the opportunity to undertake this project and encouragement in completion of this project.

We hereby wish to express a deep sense of gratitude to **Dr. B. Lakshmi Narayana Reddy,** department of CSE (AI & ML), Sri Venkateswara College of Engineering for the esteemed guidance, moral support and invaluable advice provided by him for the success of the project.

We are also thankful to all the staff members of the CSE (AI & ML) department who have co-operated in making our project a success. We would like to thank all our parents and friends who extended their help, encouragement and moral support either directly or indirectly in our project work.

Thanks for Your Valuable Guidance and kind support.

DECLARATION

We hereby declare that project report entitled **Malicious URL Detection Using Machine Learning Techniques** is a genuine project work carried out by us, in B-Tech in CSE (AI&ML) degree course of **Jawaharlal Nehru Technological University Anantapur** and has not been submitted to any other courses or University for award of any degree by us.

Signature of the Student

1. CH. NAVYA
2. S. SAMREEN SULTANA
3. M. SRAVAN KUMAR
4. G. DIVAKAR REDDY

ABSTRACT

Currently, the risk of network information insecurity is increasing rapidly in number and level of danger. Domain phishing is a scam to trick email recipients into handing over their account details via links in emails posing as their registrar. A malicious URL is a link created with the purpose of promoting scams, attacks, and frauds. When clicked on, malicious URLs can download ransomware, lead to phishing or spear phishing emails, or cause other forms of cybercrime. The methods mostly used by hackers today are to attack end-to-end technology and exploit human vulnerabilities. These techniques include social engineering, phishing, pharming, etc. One of the steps in conducting these attacks is to deceive users with malicious Uniform Resource Locators (URLs). As a results, malicious URLs and phishing detection is of great interest nowadays. There have been several scientific studies showing a number of methods to detect malicious URLs and phishing based on machine learning and deep learning techniques. In this paper, we propose a malicious URL and phishing URL detection method using machine learning techniques based on our proposed URL behaviors and attributes. Moreover, bigdata technology is also exploited to improve the capability of detection malicious URLs based on abnormal behaviors. In short, the proposed detection system consists of a new set of URLs features and behaviors, a machine learning algorithm, and a bigdata technology.

LIST OF FIGURES

LIST OF TABLES

LIST OF SCREENS

LIST OF ABBREVIATIONS

ODBC	-	Open Database Connectivity
HTTP	-	Hypertext Transfer Protocol
SQL	-	Structured Query Language
URL	-	Uniform Resource Locator
DBMS	-	Database Management System
CGI	-	Common Gateway Interface

Table of Contents

1. INTRODUCTION

1.1. PROBLEM STATEMENT

The year 2020 saw peoples life being completely dependent on technology due to the global pandemic. Since digitalization became significant in this scenario, cyber criminals went on an internet crime spree. Recent reports and researches point to an increased number of security breaches that costs the victims a huge sum of money or disclosure of confidential data. Phishing is a cybercrime that employs both social engineering and technical subterfuge in order to steal personal identity data or financial account credentials of victims. In phishing, attackers counterfeit trusted websites and misdirect people to these websites, where they are tricked into sharing usernames, passwords, banking or credit card details and other sensitive credentials. These phishing URLs may be sent to the consumers through email, instant message or text message. According to the FBI crime report 2020, phishing was the most common type of cyber attack in 2020 and phishing incidents nearly doubled from 114,702 in 2019 to 241,342 in 2020. The Verizon 2020 Data Breach Investigation Report states that 22% of data breaches in 2020 involved phishing The number of phishing attacks as observed by the Anti-Phishing Work Group (APWG) grew through 2020, doubling.

Malicious URL?

These type of URLs inject malware into the victim's system once he/she visit such URLs. Modified or compromised URLs employed for cyber attacks are known as malicious URLs.A malicious URL or website generally contains different types of trojans, malware, unsolicited content in the form of phishing, drive-by-download, spams. The main objective of the malicious website is to fraud or steal the personal or financial details of unsuspecting users. Due to the ongoing COVID-19 pandemic the incidents of cybercrime increased manifold. According to ***Symantec Internet Security Threat Report (ISTR) 2019***, malicious URLs are a highly used technique in cyber crimes.

Phishing Detection

A URL based phishing attack is carried out by sending malicious links, that seems legitimate to the users, and tricking them into clicking on it. In phishing detection, an incoming URL is identified as phishing or not by analysing the different features of the URL and is classified accordingly. Different machine learning algorithms are trained on various datasets of URL features to classify a given URL as phishing or legitimate.

Phishing Detection Approaches

In List Based approach, there are two lists, called whitelist and blacklist to classify legitimate and phishing URLs respectively. In access to websites takes place only if the URL is in the whitelist. In blacklist is used. In Heuristic Based approach, the structure of a phishing URL is analysed. A pattern of URLs that were previously classified as phishing is created. URLs are classified according to their compliance with this pattern. The methods used to process the features of the URL plays a significant role in classifying websites accurately. Visual similarity Based approach works by comparing the visual similarity of the website pages. Websites are classified as phishing or not by taking a server side view of them as in. These two data are then compared with image processing techniques. Fake web pages are designed very close to the original ones and it is easier to notice minor differences with image processing techniques, as users cannot notice them easily. Content Based approach analyses the pages content. This method extracts features from page contents and third-party services like search engines and DNS servers. In authors proposed a detection method by specifying weights to the words that draw out from URLs and HTML contents. The words might include brand names that attackers use in the URL to make it look like a real one. Weights are specified according to their presence at different positions in URLs. The most probable words are chosen and then sent to Yahoo search to return the domain name with the highest frequency between the top 30 outcomes. The owners of the domain name are compared to decide if the website is phishing or not. In they utilized a logo image to find the identity of web pages by matching real and fake web pages.

1.2. OBJECTIVES

- The objective of the project is to classify URLs based on the features and behaviors of URLs.
- Protecting the user network from malicious attacks.
- Prevents users from unauthorized access to the network.
- Detecting malicious URLs with machine learning to protect users from cyber threats.

2. LITERATURE SURVEY

2.1. RELATED STUDY

A literature survey or literature review is the study of references and old algorithms that we have read for designing the proposed methods. It also helps in reporting summarization of all the old references papers, and their drawbacks. The detailed literature survey for the project helps in comparing and contrasting various methods, algorithms in various ways that have implemented in the research.

[1] H. Rathore, A. Nandanwar, S. K. Sahay, and M. Sewak, ‘‘Adversarial superiority in Android malware detection: Lessons from reinforcement learning based evasion attacks and defenses,’’ Forensic Sci. Int., Digit. Invest., vol. 44, Mar. 2023, Art. no. 301511

Today, android smartphones are being used by billions of users and thus have become a lucrative target of malware designers. Therefore being one step ahead in this zero-sum game of malware detection between the anti-malware community and malware developers is more of a necessity than a desire. This work focuses on a proactive adversary-aware framework to develop adversarially superior android malware detection models. We first investigate the adversarial robustness of thirty-six distinct malware detection models constructed using two static features (permission and intent) and eighteen classification algorithms. We designed two Targeted Type-II Evasion Attacks (TRPO-MalEAttack and PPO-MalEAttack) based on reinforcement learning to exploit vulnerabilities in the above malware detection models. The attacks aim to add minimum perturbations in each malware application and convert it into an adversarial application that can fool the malware detection models. The TRPO-MalEAttack achieves an average fooling rate of 95.75% (with 2.02 mean perturbations), reducing the average accuracy from 86.01% to 49.11% in thirty-six malware detection models. On the other hand, The PPO-MalEAttack achieves a higher average fooling rate of 96.87% (with 2.08 mean perturbations), reducing the average accuracy from 86.01% to 48.65% in the same thirty-six detection models. We also develop a list of the TEN most vulnerable android permissions and intents that an adversary can use to generate more adversarial applications. Later, we propose a defense strategy (MalVPatch) to counter the adversarial attacks on malware detection models. The MalVPatch defense achieves higher detection accuracy along with a drastic improvement in the adversarial robustness of malware detection models. Finally, we conclude that

investigating the adversarial robustness of models is necessary before their real-world deployment and helps achieve adversarial superiority in android malware detection

[2] H. Wang, W. Zhang, and H. He, ‘‘You are what the permissions told me!Android malware detection based on hybrid tactics,’’ J. Inf. Secur. Appl., vol. 66, May 2022, Art. no. 103159.

Recent years have witnessed a significant increase in the use of Android devices in many aspects of our life. However, users can download Android apps from third-party channels, which provides numerous opportunities for malware. Attackers utilize unsolicited permissions to gain access to the sensitive private intelligence of users. Since signature-based antivirus solutions no longer meet practical needs, efficient and adaptable solutions are desperately needed, especially in new variants. As a remedy, we propose a hybrid Android malware detection approach that combines dynamic and static tactics. We firstly adopt static analysis inferring different permission usage patterns between malware and benign apps based on the machine-learning-based method. To classify the suspicious apps further, we extract the object reference relationships from the memory heap to construct a dynamic feature base. We then present an improved state-based algorithm based on DAMBA. Experimental results on a real-world dataset of 21,708 apps show that our approach outperforms the well-known detector with 97.5% F1-measure. Besides, our system is demonstrated to resist permission abuse behaviors and obfuscation techniques.

[3] A. Albakri, F. Alhayan, N. Alturki, S. Ahamed, and S. Shamsudheen, ‘‘Metaheuristics with deep learning model for cybersecurity and Android malware detection and classification,’’ Appl. Sci., vol. 13, no. 4, p. 2172, Feb. 2023.

Since the development of information systems during the last decade, cybersecurity has become a critical concern for many groups, organizations, and institutions. Malware applications are among the commonly used tools and tactics for perpetrating a cyberattack on Android devices, and it is becoming a challenging task to develop novel ways of identifying them. There are various malware detection models available to strengthen the Android operating system against such attacks. These malware detectors categorize the target applications based on the patterns that exist in the features present in the Android applications. As the analytics data continue to grow, they negatively affect the Android defense mechanisms. Since large numbers of unwanted features create a performance bottleneck for the detection mechanism, feature selection techniques are found to be

beneficial. This work presents a Rock Hyrax Swarm Optimization with deep learning-based Android malware detection (RHSODL-AMD) model. The technique presented includes finding the Application Programming Interfaces (API) calls and the most significant permissions, which results in effective discrimination between the good ware and malware applications. Therefore, an RHSO based feature subset selection (RHSO-FS) technique is derived to improve the classification results. In addition, the Adamax optimizer with attention recurrent autoencoder (ARAE) model is employed for Android malware detection. The experimental validation of the RHSODL-AMD technique on the Andro-AutoPsy dataset exhibits its promising performance, with a maximum accuracy of 99.05%

3. ANALYSIS

3.1. EXISTING SYSTEM

A popular approach in detecting malicious activity on the web is by leveraging distinguishing features between malicious and benign DNS usage. Both passive DNS monitoring and active DNS probing methods have been used to identify malicious domains. While some of these efforts focused solely on detecting fast flux service networks, another can also detect domains implementing phishing and drive-by-downloads. The best-known non-proprietary content-based approach to detect phishing webpages is Cantina

3.1.1 Limitations in Existing System

- Existing tools such as Google Safe Browsing are not enabled on the mobile versions of browsers, thereby precluding mobile users.
- DNS based mechanisms do not provide deeper understanding of the specific activity implemented by a webpage or domain.
- Downloading and executing each webpage impacts performance and hinders scalability of dynamic approaches.
- URL-based techniques usually suffer from high false positive rates.
- Cantina suffers from performance problems due to the time lag involved in querying the Google search engine. Moreover, Cantina does not work well on webpages written in languages other than English.
- Finally, existing techniques do not account for new mobile threats such as known fraud phone numbers that attempt to trigger the dialer on the phone.

3.2. PROPOSED SYSTEM

In the proposed system, machine learning algorithms are used to classify URLs based on the features and behaviors of URLs. The features are extracted from static and dynamic behaviors of URLs and are new to the literature. Those newly proposed features are the main contribution of the research. Machine learning algorithms are a part of the whole malicious and phishing URL detection system. Two supervised machine learning algorithms are used, Support vector machine (SVM) and Random forest (RF).

3.2.1. Features of the Proposed System

- Protection from malicious attacks on your network.
- Deletion and/or guaranteeing malicious elements within a preexisting network.
- Prevents users from unauthorized access to the network.
- Deny's programs from certain resources that could be infected.
- Securing confidential information
- The proposed algorithms are suitable to utilized the usefulness of our new features selected for malicious URL detection.
- In the proposed work, SVM and RF are selected as an example to illustrate the good performance of the whole detection system, and are not our main focus. Readers are encouraged to implement some other algorithms such as Naïve Bayes, Decision trees, k-nearest neighbors, neural networks, etc.

ALGORITHMS

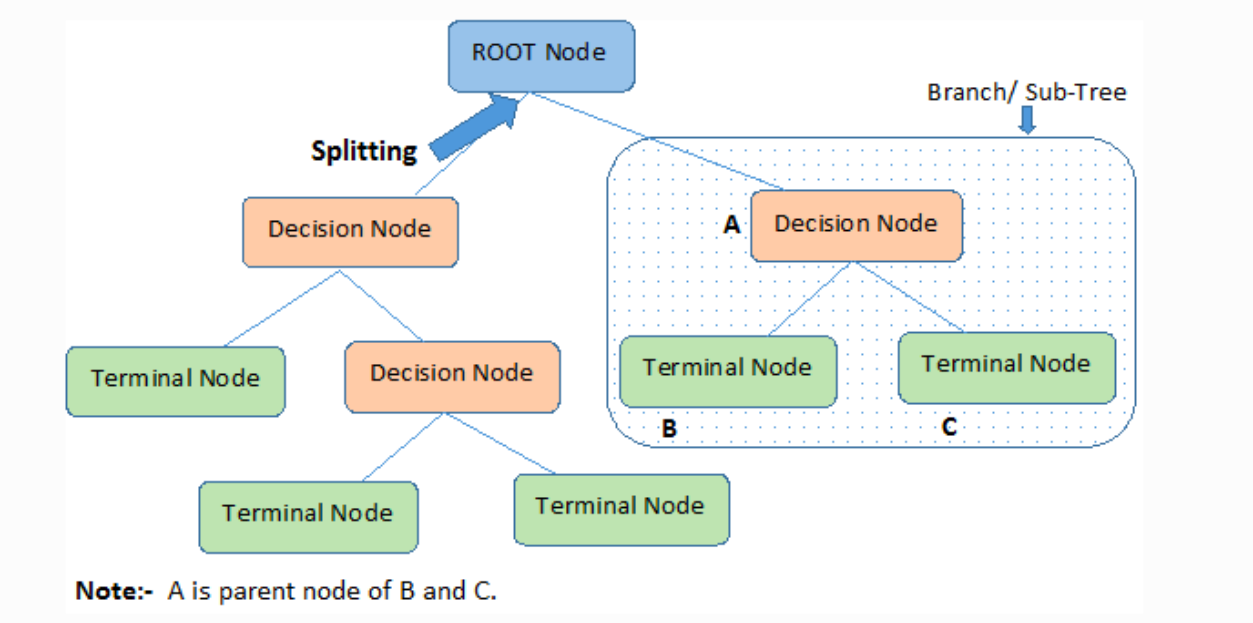
Decision tree classifiers

Decision tree classifiers are widely applied across various domains due to their ability to capture descriptive decision-making knowledge from provided data. One of their most notable features is their capability to be generated from training sets. The process for generating decision trees based on a set of objects (S), each belonging to one of the classes C1, C2, …, Ck, unfolds as follows:

Step 1: If all objects in S belong to the same class (e.g., Ci), the decision tree for S comprises a leaf labeled with this class.

Step 2: Otherwise, select a test (T) with potential outcomes (O1, O2,…, On). Each object in S has a specific outcome for test T, thus partitioning S into subsets (S1, S2,…, Sn), where each object in Si corresponds to outcome Oi for test T. Test T becomes the root of the decision tree, and for each outcome Oi, a subsidiary decision tree is constructed by recursively invoking the same procedure on set Si.

Block Diagram for Decision Tree Algorithm:



Root Node: It represents the entire population or sample and this further gets divided into two or more homogeneous sets.

Splitting: It is a process of dividing a node into two or more sub-nodes.

Decision Node: When a sub-node splits into further sub-nodes, then it is called the decision node.

Leaf / Terminal Node: Nodes do not split is called Leaf or Terminal node.

Pruning: When we remove sub-nodes of a decision node, this process is called pruning. You can say the opposite process of splitting.

Branch / Sub-Tree: A subsection of the entire tree is called branch or sub-tree.

Gradient boosting

Gradient boosting is a versatile machine learning technique employed in regression and classification tasks, among others. It constructs a prediction model in the form of an ensemble of weak prediction models, typically decision trees. When using decision trees as the weak learner, the resulting algorithm is referred to as gradient-boosted trees, often surpassing the performance of random forests. The construction of a gradient-boosted trees model occurs in a stage-wise manner, similar to other boosting methods, but it stands out by allowing the optimization of an arbitrary differentiable loss function.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a straightforward yet highly effective classification algorithm that operates based on a similarity measure. It is non-parametric and employs lazy learning, meaning it does not "learn" until presented with a test example. Whenever there is a new data point to classify, KNN identifies its K-nearest neighbors from the training data and determines its classification based on their majority vote or weighted vote.

Logistic regression

Logistic regression analysis explores the relationship between a categorical dependent variable and a set of independent variables. The term "logistic regression" is applied when the dependent variable has only two values, such as 0 and 1, or Yes and No. On the other hand, "multinomial logistic regression" is used when the dependent variable has three or more unique values, like Married, Single, Divorced, or Widowed. While the nature of data for the dependent variable differs from that of multiple regressions, the practical application of the procedure remains similar.

Logistic regression serves as a competitor to discriminant analysis in analyzing categorical-response variables. Many statisticians favor logistic regression due to its versatility and suitability for modeling various situations compared to discriminant analysis. This preference arises from logistic regression's ability to not assume that the independent variables follow a normal distribution, unlike discriminant analysis.

NAIVE BAYES

The naive Bayes approach is a supervised learning method founded on a simple assumption: it presumes that the presence or absence of one feature of a class is

independent of the presence or absence of any other feature. Despite its simplicity, it demonstrates robustness and efficiency comparable to other supervised learning techniques. One explanation often highlighted in the literature is based on representation bias.

The naive Bayes classifier operates as a linear classifier, akin to linear discriminant analysis, logistic regression, or linear support vector machines (SVMs). However, the distinction lies in the method used to estimate the classifier's parameters, known as the learning bias. Although the naive Bayes classifier finds extensive use in the research community due to its ease of programming, parameter estimation simplicity, rapid learning even with large datasets, and reasonably good accuracy compared to other methods, it remains less popular among practitioners seeking practical results. Researchers appreciate its simplicity and efficacy. However, practitioners often struggle with its interpretability and deployment, as they may not grasp its relevance or utility.

Random forests

Random forests, also known as random decision forests, represent an ensemble learning technique used for classification, regression, and other tasks. They function by constructing numerous decision trees during training. For classification tasks, the output of the random forest is determined by the class selected by the majority of trees. Conversely, for regression tasks, the mean or average prediction of the individual trees is returned. Random decision forests aim to mitigate the issue of decision trees overfitting to their training set.

In general, random forests tend to outperform individual decision trees, although they may have lower accuracy compared to gradient boosted trees. Nonetheless, the performance of random forests can be influenced by the characteristics of the data.

The concept of random decision forests was first introduced in 1995 by Tin Kam Ho, who utilized the random subspace method. This method, as formulated by Ho, serves as an implementation of the "stochastic discrimination" approach to classification initially proposed by Eugene Kleinberg.

Support Vector Machine (SVM)

Support Vector Machine (SVM) represents a discriminant machine learning technique commonly used in classification tasks. It aims to find a discriminant function, based on an independently and identically distributed training dataset, that accurately predicts labels for newly acquired instances. Unlike generative machine learning approaches, which necessitate computations of conditional probability distributions, a discriminant classification function assigns a data point x to one of the classes involved in the classification task. Compared to generative approaches, discriminant methods may be less powerful, particularly in outlier detection scenarios. However, they require fewer computational resources and less training data, especially in multidimensional feature spaces and when only posterior probabilities are necessary. Geometrically, learning a classifier equates to identifying the equation for a multidimensional surface that optimally separates the different classes in the feature space.

SVM is a discriminant technique that solves convex optimization problems analytically, consistently yielding the same optimal hyperplane parameters. In contrast, genetic algorithms (GAs) and perceptrons, both widely used for classification in machine learning, may produce solutions highly dependent on initialization and termination criteria. With a specific kernel transforming data from the input space to the feature space, SVM training returns uniquely defined model parameters for a given training set, whereas perceptron and GA classifier models vary with each training iteration.

4. SOFTWARE AND HARDWARE SPECIFICATIONS

4.1 SOFTWARE REQUIREMENTS

Operating System	:	Windows 7/8/10
Coding Language	:	Python
Server	:	Wamp Server
Database	:	MYSQL

4.2 HARDWARE REQUIREMENTS

System	:	Pentium IV 2.4 GHz or More
Hard Disk	:	250 GB
RAM	:	4 GB

5. DESIGN

5.1. SYSTEM ARCHITECTURE

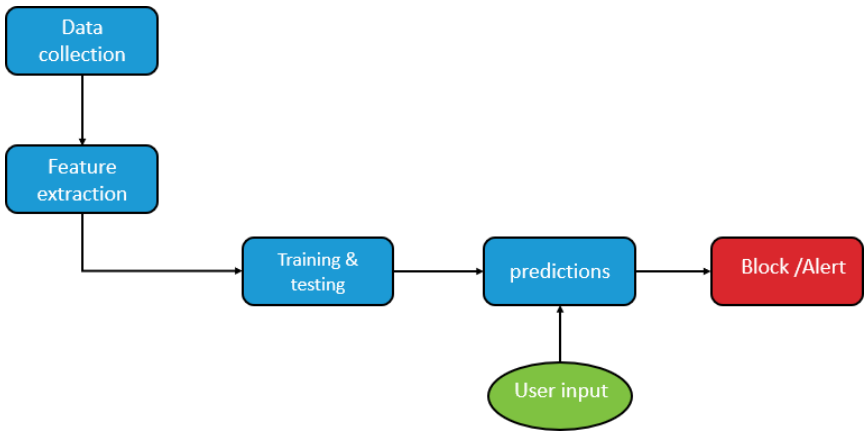


Fig 5.1: System Architecture

5.2. UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

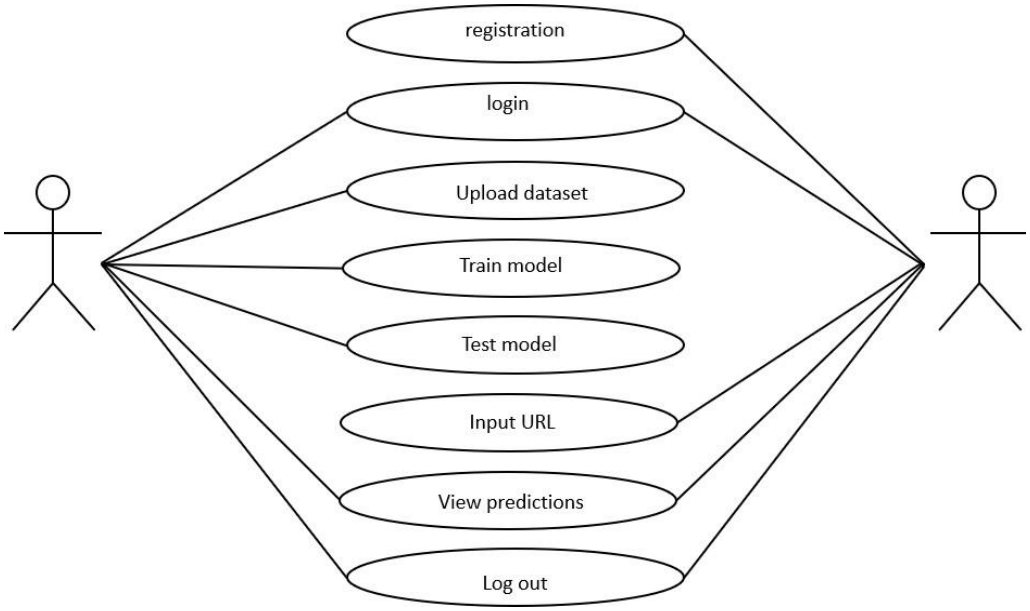
The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.

5.2.1. Use Case Diagram

A use case diagram is a type of behavioural diagram created from a Use-case analysis. The purpose of use case is to present overview of the functionality provided by the system in terms of actors, their goals and any dependencies between those use cases. In the below diagram the use cases are depicted with actors and their relationships.

Fig.5.2: Use Case Diagram for Overall Project



5.2.2. Class Diagram

A class diagram in the UML is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, and the relationships between the classes. Private visibility hides information from anything outside the class partition. Public visibility allows all other classes to view the marked information. Protected visibility allows child classes to access information they inherited from a parent class.

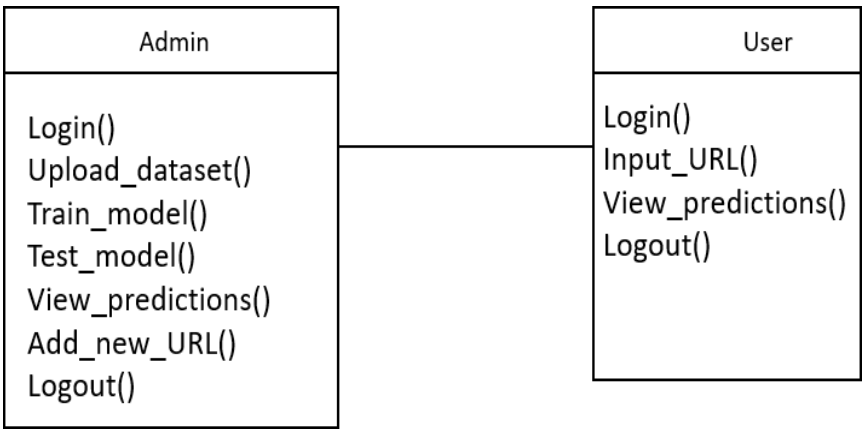


Fig.5.3: Class Diagram for Overall Project

5.2.3. Sequence Diagram

The sequence diagram describes the flow of messages being passed from object to object. Unlike the class diagram, the sequence diagram represents dynamic message passing between instances of classes rather than just a static structure of classes. In some ways, a sequence diagram is like a stack trace of object messages.

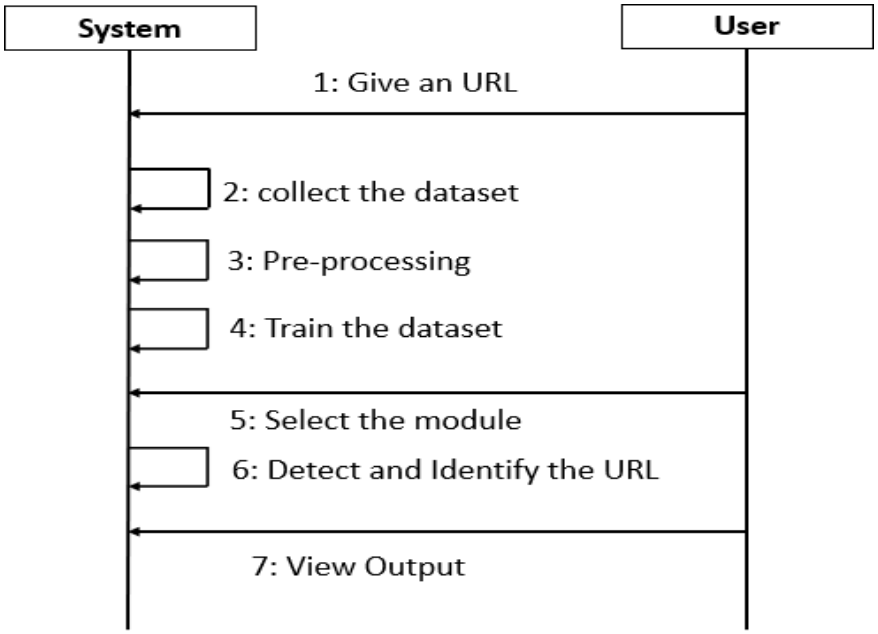


Fig.5.4: Sequence Diagram for Overall Project

5.2.4. Activity Diagram

Activity Diagram in some ways is like a flowchart with states. With the activity diagram you can follow flow of activities in your system in the order that they take place. An activity diagram illustrates the dynamic nature of a system by modelling the flow of control from activity to activity. Because an activity diagram is a special kind of state chart diagram, it uses some of the same modelling conventions.

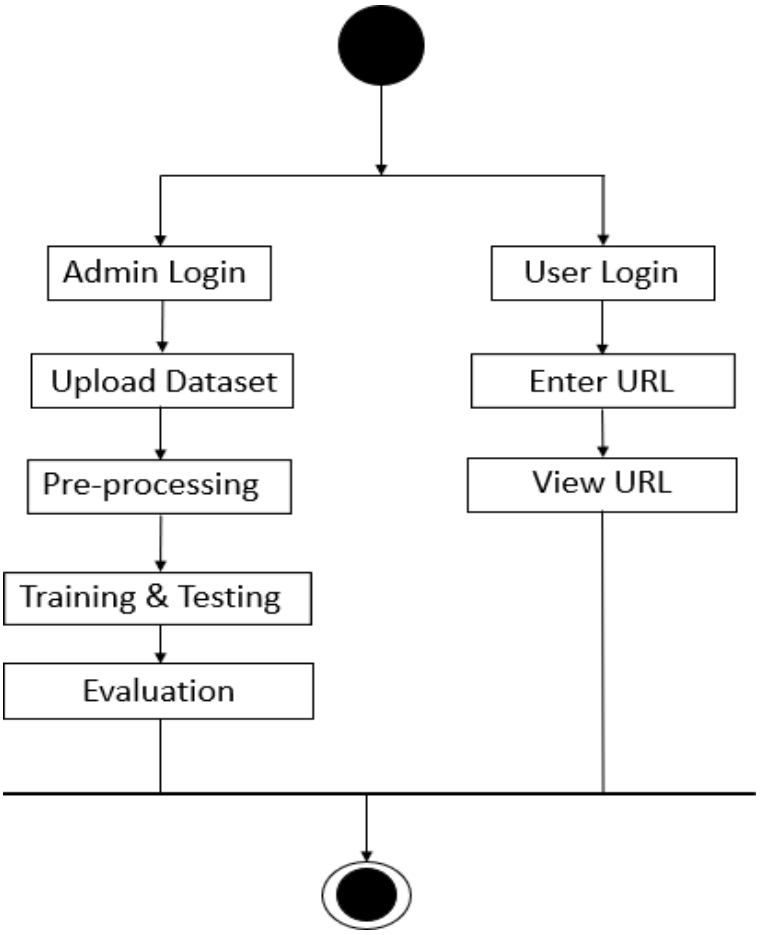


Fig.5.5: Activity Diagram for Client

5.2.5. Deployment Diagram

A Deployment Diagram in system architecture represents the physical deployment of software components on hardware nodes. It shows how system components interact in a real-world environment. Key elements include nodes (servers, devices), artifacts (software components), and communication paths (connections between nodes). Common deployment architectures include on-premises, cloud-based, and hybrid models. It ensures scalability, performance, and security of the deployed system. Properly designed deployment diagrams aid in efficient system maintenance and upgrades.

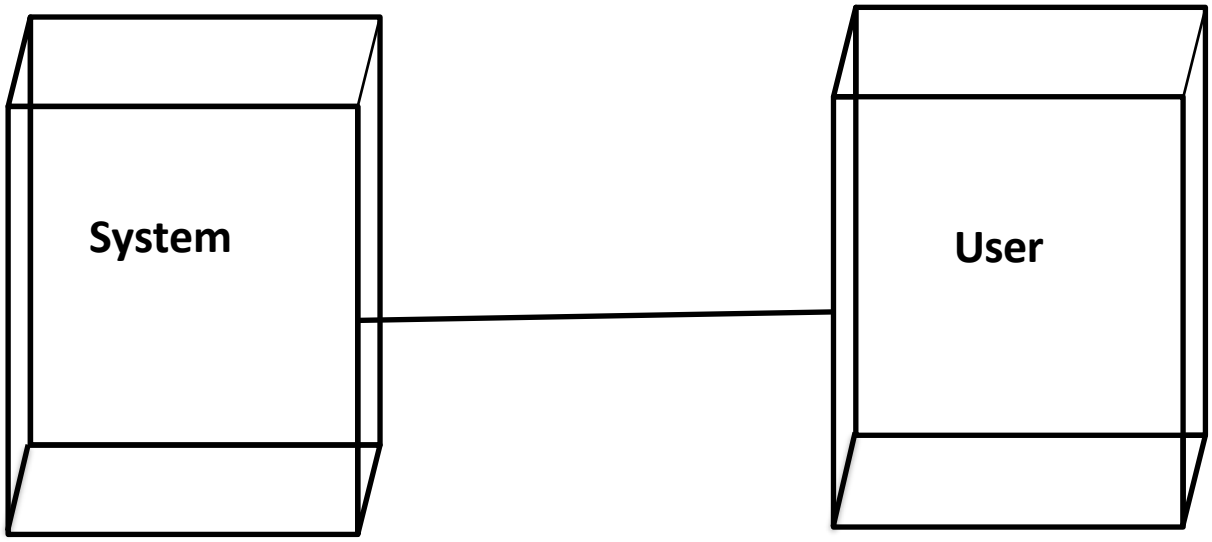


Fig.5.6: Deployment Diagram

5.2.6 Data Flow Diagrams

The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.

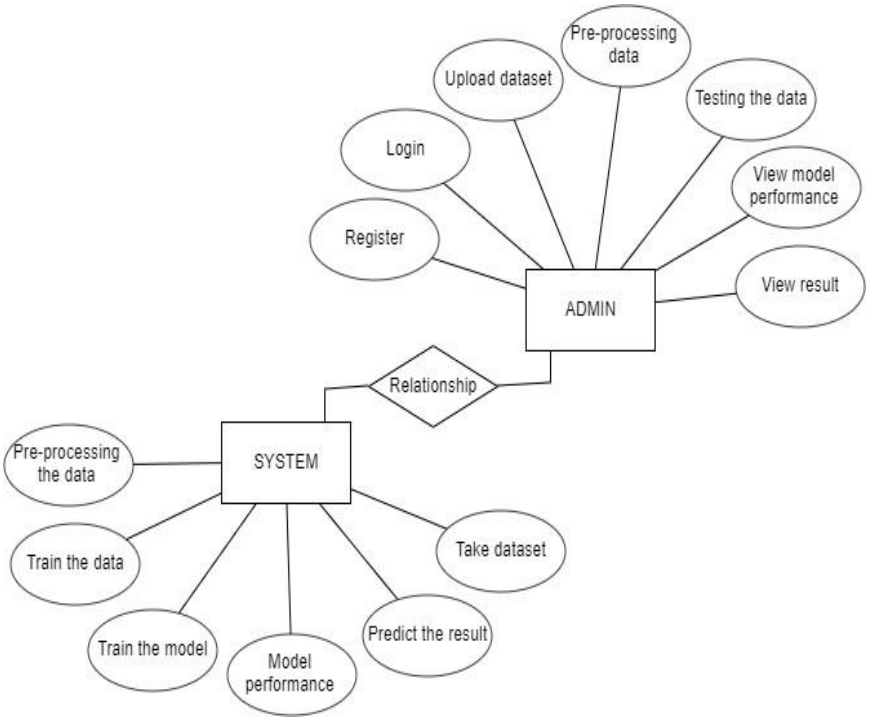


Fig.5.7: Data Flow Diagram

5.3 MODULES

5.3.1 Data Collection

- The dataset for Malicious URL prediction is collected from the Kaggle website.
- we collected sufficient data samples and legitimate software samples.
- We split the data into Training and Testing, where the Training dataset is for training the model Testing is for evaluating the model.

5.3.2 Data Processing

- Once data is gathered, it needs to be pre-processed, cleaned, constructed, and formatted in a style that SVM comprehends and is able to work with.
- Data Processing techniques Provide cleaning of Data Samples that will be used for better performance.
- After data is preprocessed, it is added to the Malicious URL dataset, which can be used for training and testing purposes.

5.3.3 Training and Testing

- We Split the data into train and test data Train will be used for training the model and Test data to check the performance
- The ratio of 80:20 is a common split in Machine Learning because it provides enough data for training while keeping a sufficient portion for testing.
- We use the preprocessed training dataset to train our model using SVM Algorithm.

5.3.4 Evaluate and Detect Malicious URL's

- Models are evaluated using classification reports and confusion matrices, like precision, recall, and F1 score and measure the accuracy by identifying the Malicious URL accurately.

- Based on the model trained algorithm we will detect whether the given URL is Phishing or Malicious.

6. IMPLEMENTATION

6.1. Python

Below are some facts about Python. Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc. The biggest strength of Python is huge collection of standard library which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python :-

Let’s see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don’t have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.

6.1.1 History of Python

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python.Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners1, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it."Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that

possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

What is Machine Learning : -

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of building models of data. Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models tunable parameters that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain.Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Leaning :-

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning. Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into classification tasks and regression tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section. Unsupervised learning involves modeling the features of a dataset

without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as clustering and dimensionality reduction. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale". Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programing logic, in the problems that cSVM & NAVI BAYESot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machines Learning :-

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are −

Quality of data − Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task − Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons − As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems − Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting − If the model is overfitting or underfitting, it SVM & NAVI BAYE be represented well for the problem.

Curse of dimensionality − Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment − Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machines Learning :-

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which SVM & NAVI BAYE be solved with traditional approach. Following are some real-world applications of ML −

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition

- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

6.1.2 Learning Machine Learning

Arthur Samuel coined the term "Machine Learning" in 1959 and defined it as a "Field of study that gives computers the capability to learn without being explicitly programmed".

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to Indeed, Machine Learning Engineer Is The Best Job of 2019 with a 344% growth and an average base salary of $146,085 per year.

But there is still a lot of doubt about what exactly is Machine Learning and how to start learning it? So this article deals with the Basics of Machine Learning and also the path you can follow to eventually become a full-fledged Machine Learning Engineer. Now let's get started!!!

How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that

heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

7. SOURCE CODE

7.1 SAMPLE SOURCE CODE

VIEW.PY

```
from django.db.models import  Count, Avg
from django.shortcuts import render, redirect
from django.db.models import Count
from django.db.models import Q
import datetime
import xlwt
from django.http import HttpResponse
import nltk
import re
import string
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer
from nltk.stem.wordnet import WordNetLemmatizer
import pandas as pd
from wordcloud import WordCloud, STOPWORDS
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import accuracy_score
from sklearn.metrics import f1_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
# Create your views here.

from Remote_User.models import
ClientRegister_Model,detecting_Type,detection_ratio,detection_accuracy


def serviceproviderlogin(request):

    if request.method  == "POST":

        admin = request.POST.get('username')

        password = request.POST.get('password')

        if admin == "Admin" and password =="Admin":

            detection_accuracy.objects.all().delete()

            return redirect('View_Remote_Users')


    return render(request,'SProvider/serviceproviderlogin.html')


def View_Website_URL_Type_Ratio(request):

    detection_ratio.objects.all().delete()

    ratio = ""

    kword = 'Phishing'

    print(kword)

    obj = detecting_Type.objects.all().filter(Q(Prediction=kword))

    obj1 = detecting_Type.objects.all()

    count = obj.count();

    count1 = obj1.count();

    ratio = (count / count1) * 100

    if ratio != 0:

        detection_ratio.objects.create(names=kword, ratio=ratio)
```

```
    ratio1 = ""
    kword1 = 'Non Phishing'
    print(kword1)
    obj1 = detecting_Type.objects.all().filter(Q(Prediction=kword1))
    obj11 = detecting_Type.objects.all()
    count1 = obj1.count();
    count11 = obj11.count();
    ratio1 = (count1 / count11) * 100
    if ratio1 != 0:
        detection_ratio.objects.create(names=kword1, ratio=ratio1)


    obj = detection_ratio.objects.all()
    return render(request, 'SProvider/View_Website_URL_Type_Ratio.html', {'objs':
obj})


def View_Remote_Users(request):
    obj=ClientRegister_Model.objects.all()
    return render(request,'SProvider/View_Remote_Users.html',{'objects':obj})


def ViewTrendings(request):
    topic =
detecting_Type.objects.values('topics').annotate(dcount=Count('topics')).order_by('-
dcount')
    return  render(request,'SProvider/ViewTrendings.html',{'objects':topic})


def charts(request,chart_type):
    chart1 = detection_ratio.objects.values('names').annotate(dcount=Avg('ratio'))
```

```
    return render(request,"SProvider/charts.html", {'form':chart1, 'chart_type':chart_type})


def charts1(request,chart_type):

    chart1 = detection_accuracy.objects.values('names').annotate(dcount=Avg('ratio'))

    return render(request,"SProvider/charts1.html", {'form':chart1,
'chart_type':chart_type})


def View_Prediction_Of_Website_URL_Type(request):

    obj =detecting_Type.objects.all()

    return render(request, 'SProvider/View_Prediction_Of_Website_URL_Type.html',
{'list_objects': obj})


def likeschart(request,like_chart):

    charts =detection_accuracy.objects.values('names').annotate(dcount=Avg('ratio'))

    return render(request,"SProvider/likeschart.html", {'form':charts,
'like_chart':like_chart})



def Download_Trained_DataSets(request):

    response = HttpResponse(content_type='application/ms-excel')

    # decide file name

    response['Content-Disposition'] = 'attachment; filename="Predicted_Data.xls"'

    # creating workbook

    wb = xlwt.Workbook(encoding='utf-8')

    # adding sheet

    ws = wb.add_sheet("sheet1")

    # Sheet header, first row
```

```
    row_num = 0
    font_style = xlwt.XFStyle()
    # headers are bold
    font_style.font.bold = True
    # writer = csv.writer(response)
    obj = detecting_Type.objects.all()
    data = obj  # dummy method to fetch data.
    for my_row in data:
        row_num = row_num + 1


        ws.write(row_num, 0, my_row.Url_Text, font_style)
        ws.write(row_num, 1, my_row.Prediction, font_style)
    wb.save(response)
    return response


def train_model(request):
    detection_accuracy.objects.all().delete()
    data = pd.read_csv("Website_urls.csv")


    mapping = {'Phishing': 0, 'Non Phishing': 1}
    data['Results'] = data['Label'].map(mapping)


    x = data['URL']
    y = data['Results']
   cv = CountVectorizer()
    print(x)
    print("Y")
```

```
print(y)
x = cv.fit_transform(x)
models = []
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.20)

X_train.shape, X_test.shape, y_train.shape

print("Naive Bayes")

from sklearn.naive_bayes import MultinomialNB

NB = MultinomialNB()

NB.fit(X_train, y_train)

predict_nb = NB.predict(X_test)

naivebayes = accuracy_score(y_test, predict_nb) * 100

print("ACCURACY")

print(naivebayes)

print("CLASSIFICATION REPORT")

print(classification_report(y_test, predict_nb))

print("CONFUSION MATRIX")

print(confusion_matrix(y_test, predict_nb))

detection_accuracy.objects.create(names="Naive Bayes", ratio=naivebayes)


# SVM Model
print("SVM")

from sklearn import svm

lin_clf = svm.LinearSVC()

lin_clf.fit(X_train, y_train)

predict_svm = lin_clf.predict(X_test)

svm_acc = 94

print("ACCURACY")
```

```
print(svm_acc)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, predict_svm))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_svm))
detection_accuracy.objects.create(names="SVM", ratio=svm_acc)
print("Logistic Regression")

from sklearn.linear_model import LogisticRegression
reg = LogisticRegression(random_state=0, solver='lbfgs').fit(X_train, y_train)
y_pred = reg.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, y_pred) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, y_pred))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, y_pred))
detection_accuracy.objects.create(names="Logistic Regression", ratio=93)

print("Decision Tree Classifier")
dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)
dtcpredict = dtc.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, dtcpredict) * 100)
print("CLASSIFICATION REPORT")
```

```
print(classification_report(y_test, dtcpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, dtcpredict))
detection_accuracy.objects.create(names="Decision Tree Classifier", ratio=91)

print("Random Forest Classifier")
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
pred_rfc = rf.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, pred_rfc) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, pred_rfc))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, pred_rfc))
detection_accuracy.objects.create(names="Random Forest Classifier", ratio=88)

print("SGD Classifier")
from sklearn.linear_model import SGDClassifier
sgd_clf = SGDClassifier(loss='hinge', penalty='l2', random_state=0)
sgd_clf.fit(X_train, y_train)
sgdpredict = sgd_clf.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, sgdpredict) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, sgdpredict))
```

```
    print("CONFUSION MATRIX")
    print(confusion_matrix(y_test, sgdpredict))
    detection_accuracy.objects.create(names="SGD Classifier", ratio=87)
labeled = 'labeled_data.csv'
    data.to_csv(labeled, index=False)


    obj = detection_accuracy.objects.all()
          return render(request,'SProvider/train_model.html', {'objs': obj})
```

Remote_user

```
from django.db.models import Count

from django.db.models import Q

from django.shortcuts import render, redirect, get_object_or_404

import datetime

import openpyxl


import nltk

import re

import string

from nltk.corpus import stopwords

from sklearn.feature_extraction.text import CountVectorizer

from nltk.stem.wordnet import WordNetLemmatizer

from sklearn.ensemble import RandomForestClassifier

import pandas as pd

from wordcloud import WordCloud, STOPWORDS

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from sklearn.metrics import accuracy_score

from sklearn.metrics import f1_score

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import VotingClassifier

# Create your views here.

from Remote_User.models import
ClientRegister_Model,detecting_Type,detection_ratio,detection_accuracy
```

```
def login(request):
if request.method == "POST" and 'submit1' in request.POST:
username = request.POST.get('username')
        password = request.POST.get('password')
        try:
            enter = ClientRegister_Model.objects.get(username=username,password=password)
            request.session["userid"] = enter.id


            return redirect('ViewYourProfile')
        except:
            pass


    return render(request,'RUser/login.html')
def Add_DataSet_Details(request):


    return render(request, 'RUser/Add_DataSet_Details.html', {"excel_data": ''})
def Register1(request):


    if request.method == "POST":
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        phoneno = request.POST.get('phoneno')
        country = request.POST.get('country')
        state = request.POST.get('state')
        city = request.POST.get('city')
```

```
        ClientRegister_Model.objects.create(username=username, email=email,
password=password, phoneno=phoneno,
                                            country=country, state=state, city=city)
            return render(request, 'RUser/Register1.html')
    else:
        return render(request,'RUser/Register1.html')


def ViewYourProfile(request):
    userid = request.session['userid']
    obj = ClientRegister_Model.objects.get(id= userid)
    return render(request,'RUser/ViewYourProfile.html',{'object':obj})


def Predict_Website_URL_Type(request):
    if request.method == "POST":
        url_text = request.POST.get('keyword')
        if request.method == "POST":
            url_text = request.POST.get('keyword')


        data = pd.read_csv("Website_urls.csv")


        mapping = {'Phishing': 0, 'Non Phishing': 1}
        data['Results'] = data['Label'].map(mapping)


        x = data['URL']
        y = data['Results']


        cv = CountVectorizer()
```

```
print(x)
print("Y")
print(y)
x = cv.fit_transform(x)

models = []
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.20)
X_train.shape, X_test.shape, y_train.shape

print("Naive Bayes")

from sklearn.naive_bayes import MultinomialNB

NB = MultinomialNB()
NB.fit(X_train, y_train)
predict_nb = NB.predict(X_test)
naivebayes = accuracy_score(y_test, predict_nb) * 100
print(naivebayes)
print(confusion_matrix(y_test, predict_nb))
print(classification_report(y_test, predict_nb))
models.append(('naive_bayes', NB))

# SVM Model
print("SVM")
from sklearn import svm
```

```
lin_clf = svm.LinearSVC()
lin_clf.fit(X_train, y_train)
predict_svm = lin_clf.predict(X_test)
svm_acc = accuracy_score(y_test, predict_svm) * 100
print(svm_acc)
print("CLASSIFICATION  REPORT")
print(classification_report(y_test, predict_svm))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_svm))
models.append(('svm', lin_clf))

print("Logistic  Regression")

from sklearn.linear_model import LogisticRegression

reg = LogisticRegression(random_state=0, solver='lbfgs').fit(X_train, y_train)
y_pred = reg.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, y_pred) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, y_pred))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, y_pred))
models.append(('logistic', reg))

print("Decision Tree Classifier")
```

```
dtc = DecisionTreeClassifier()

dtc.fit(X_train, y_train)

dtcpredict = dtc.predict(X_test)

print("ACCURACY")

print(accuracy_score(y_test, dtcpredict) * 100)

print("CLASSIFICATION REPORT")

print(classification_report(y_test, dtcpredict))

print("CONFUSION MATRIX")

print(confusion_matrix(y_test, dtcpredict))

models.append(('DecisionTreeClassifier', dtc))


print("SGD Classifier")

from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(loss='hinge', penalty='l2', random_state=0)

sgd_clf.fit(X_train, y_train)

sgdpredict = sgd_clf.predict(X_test)

print("ACCURACY")

print(accuracy_score(y_test, sgdpredict) * 100)

print("CLASSIFICATION REPORT")

print(classification_report(y_test, sgdpredict))

print("CONFUSION MATRIX")

print(confusion_matrix(y_test, sgdpredict))

models.append(('SGDClassifier', sgd_clf))


rf = RandomForestClassifier()

rf.fit(X_train, y_train)

pred_rfc = rf.predict(X_test)
```

```
print("ACCURACY")
print(accuracy_score(y_test, pred_rfc) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, pred_rfc))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test,  pred_rfc))

classifier = VotingClassifier(models)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

url_text1 = [url_text]
vector1 = cv.transform(url_text1).toarray()
predict_text = classifier.predict(vector1)

pred = str(predict_text).replace("[", "")
pred1 = str(pred.replace("]", ""))

prediction = int(pred1)

if prediction == 0:
   val = 'Phishing'
elif prediction == 1:
   val = 'Non Phishing'

print(prediction)
print(val)
```

```
        detecting_Type.objects.create(Url_Text=url_text,  Prediction=val)


        return render(request, 'RUser/Predict_Website_URL_Type.html',{'objs': val})
    return render(request, 'RUser/Predict_Website_URL_Type.html')
```

8. TESTING AND RESULTS

8.1. SOFTWARE TESTING TECHNIQUES

Software testing is a critical element of software quality assurance and represents the ultimate review of specification, designing and coding.

8.1.1. Unit Testing

In software testing, Unit testing mainly focuses on verification effort on the smallest unit of program or software design that is also called a module. In unit testing the procedural or functional design provides a detailed description as a guide, focal the control paths are tested to uncover errors occurred in the designed software within the boundaries of the module. The unit testing of software is normally white box or open testing oriented and the series of steps can be conducted in corresponding or parallel for multiple modules or functions.

8.1.2. Integration Testing

Integration testing is another Testing for systematic technique and product module integrating which constructs the program structure and makes the data flow between the modules, while conducting Integration Testing it requires to uncover errors associated with various interfaces. The main objective is to take unit tested methods and activities to build a program structure that have been dictated by design.

8.1.3. Validation Testing

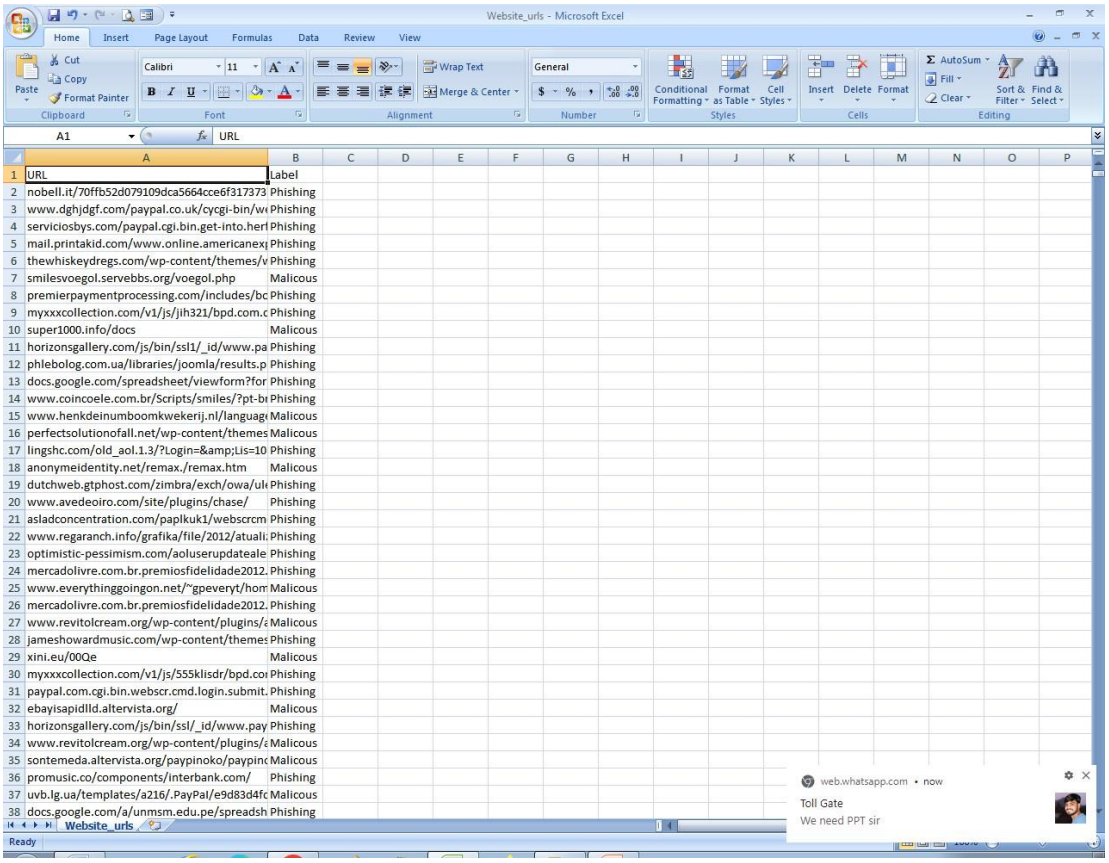
The Validation Testing is integration testing for software which is completely assembled as a package. The Validation testing is the next stage in Testing Activities, which can be defined as successful testing process for the software functions in the SVM & NAVI BAYE reasonably expected by the customer. The validation Testing is mainly performed at the end approach of the user needs in testing the information inputed to the product and information contained in those sections are to validated through various testing approaches.

8.2. TEST CASES

S. No.	TEST CASES	INPUT	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	User Registration	Enter all fields	User gets registered	Registration is successful	pass
2	User Registration	if user miss any field	User not registered	Registration is un successful	failp
3	Admin Login	Give the user name and password	Admin home page should be opened	Admin home Page has been opened	Pass
4	Upload **url** Dataset	Test whether the **url** Dataset is uploaded or not into the system	If **url** Dataset is not uploaded	We cannot do further operations	Pass
5	Preprocess Dataset	Verify the Dataset is Per – processed or not	Without loading the dataset	We cannot Per-processing Dataset	Pass
6	Run SVM algorithm	Verify the SVM algorithm will run or not	Without training model	We cannot run SVM algorithm	Pass

Table 7.1: Test Case Results

9. OUTPUT SCREENSHOTS

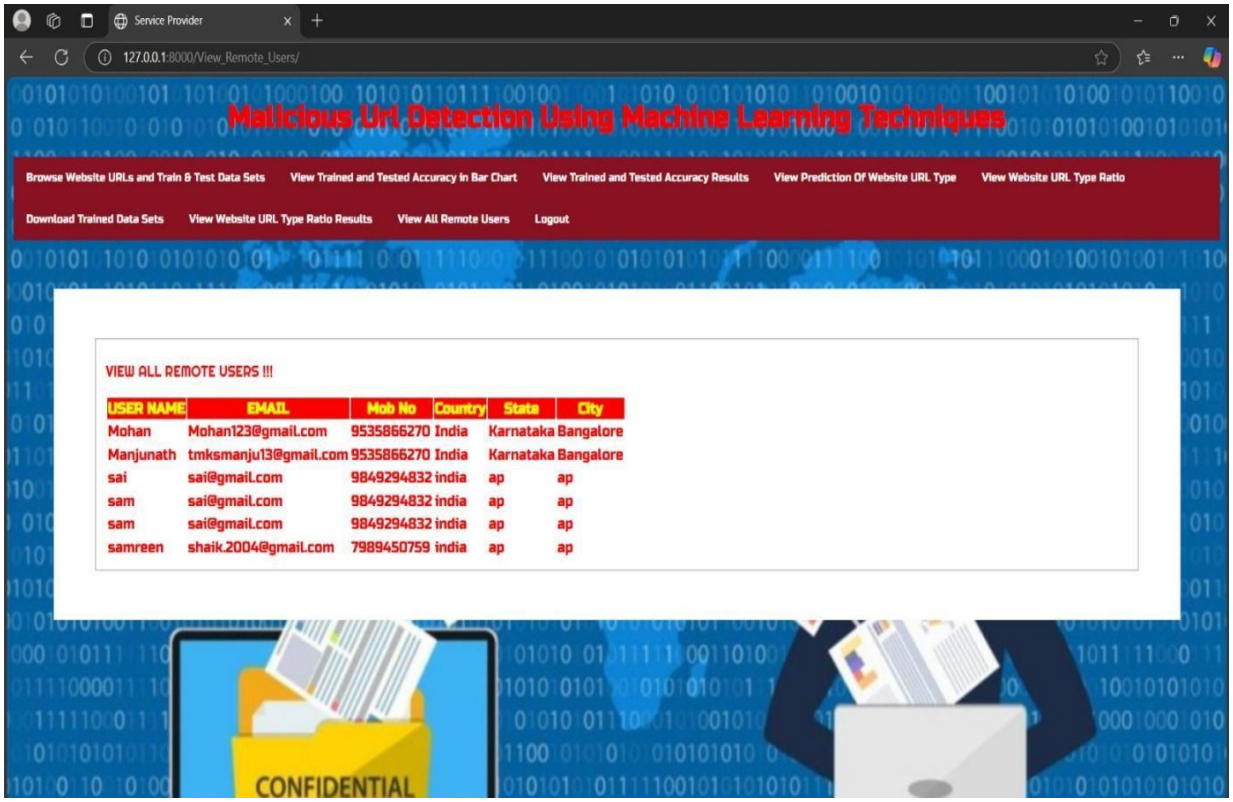


	A	B
1	URL	Label
2	nobell.it/70ffb52d079109dca5664cce6f317373	Phishing
3	www.dghjdgf.com/paypal.co.uk/cycgi-bin/we	Phishing
4	serviciosbys.com/paypal.cgi.bin.get-into.herf	Phishing
5	mail.printakid.com/www.online.americanexp	Phishing
6	thewhiskeydregs.com/wp-content/themes/v	Phishing
7	smilesvoegol.servebbs.org/voegol.php	Malicous
8	premierpaymentprocessing.com/includes/bc	Phishing
9	myxxxcollection.com/v1/js/jih321/bpd.com.c	Phishing
10	super1000.info/docs	Malicous
11	horizonsgallery.com/js/bin/ssl1/_id/www.pa	Phishing
12	phlebolog.com.ua/libraries/joomla/results.p	Phishing
13	docs.google.com/spreadsheet/viewform?for	Phishing
14	www.coincoele.com.br/Scripts/smiles/?pt-br	Phishing
15	www.henkdeinumboomkwekerij.nl/language	Malicous
16	perfectsolutionofall.net/wp-content/themes	Malicous
17	lingshc.com/old_aol.1.3/?Login=&Lis=10	Phishing
18	anonymeidentity.net/remax./remax.htm	Malicous
19	dutchweb.gtphost.com/zimbra/exch/owa/ul	Phishing
20	www.avedeoiro.com/site/plugins/chase/	Phishing
21	asladconcentration.com/paplkuk1/webscrcm	Phishing
22	www.regaranch.info/grafika/file/2012/atuali	Phishing
23	optimistic-pessimism.com/aoluserupdateale	Phishing
24	mercadolivre.com.br.premiosfidelidade2012.	Phishing
25	www.everythinggoingon.net/~gpeveryt/hom	Malicous
26	mercadolivre.com.br.premiosfidelidade2012.	Phishing
27	www.revitolcream.org/wp-content/plugins/a	Malicous
28	jameshowardmusic.com/wp-content/themes	Phishing
29	xini.eu/00Qe	Malicous
30	myxxxcollection.com/v1/js/555klisdr/bpd.co	Phishing
31	paypal.com.cgi.bin.webscr.cmd.login.submit.	Phishing
32	ebayisapidlld.altervista.org/	Malicous
33	horizonsgallery.com/js/bin/ssl/_id/www.pay	Phishing
34	www.revitolcream.org/wp-content/plugins/a	Malicous
35	sontemeda.altervista.org/paypinoko/paypin	Malicous
36	promusic.co/components/interbank.com/	Phishing
37	uvb.lg.ua/templates/a216/.PayPal/e9d83d4fc	Malicous
38	docs.google.com/a/unmsm.edu.pe/spreadsh	Phishing

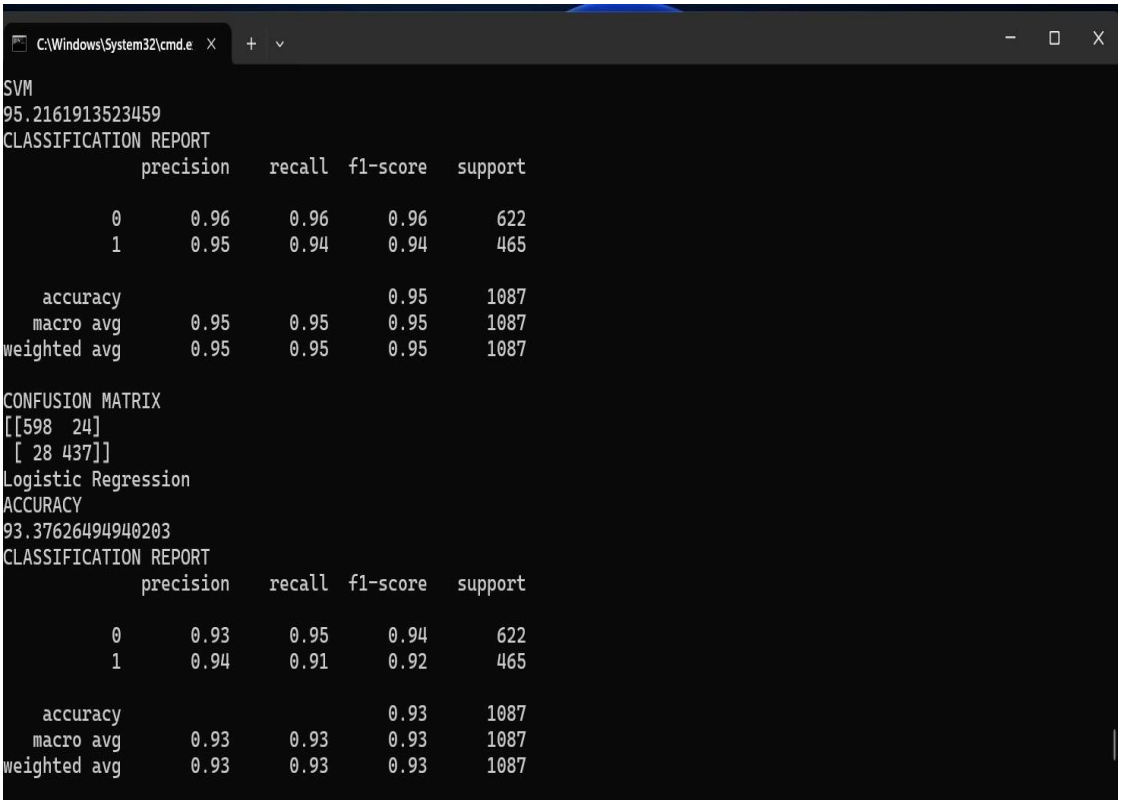
Screen 1: Data Sets

Screen 2: Home Page

Screen 3: Login Page

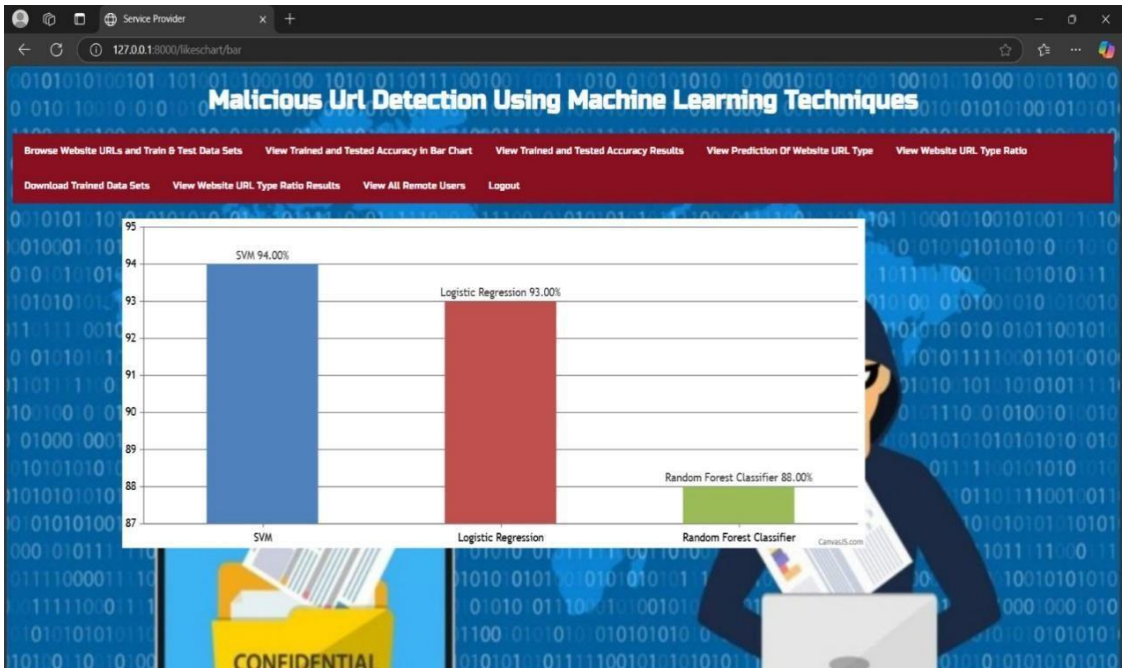


Screen 4: Admin Menu Operations

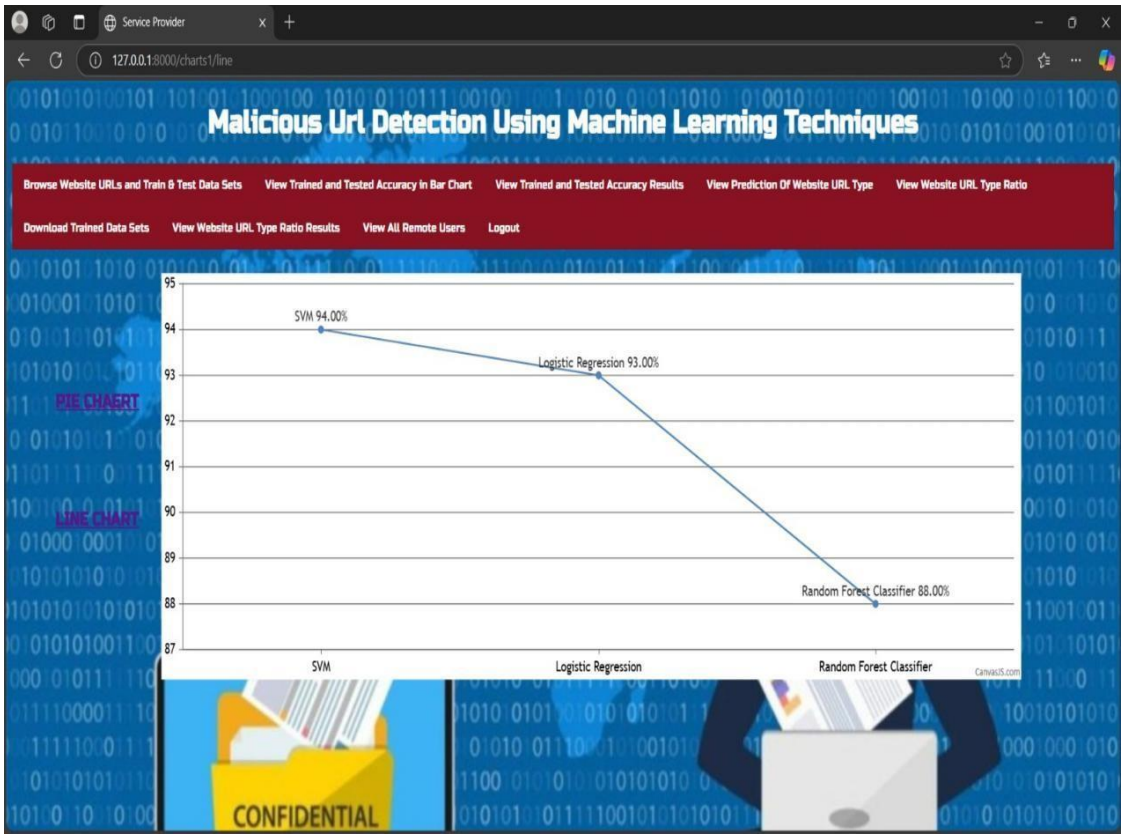


Screen 5: ML Accuracy Chart

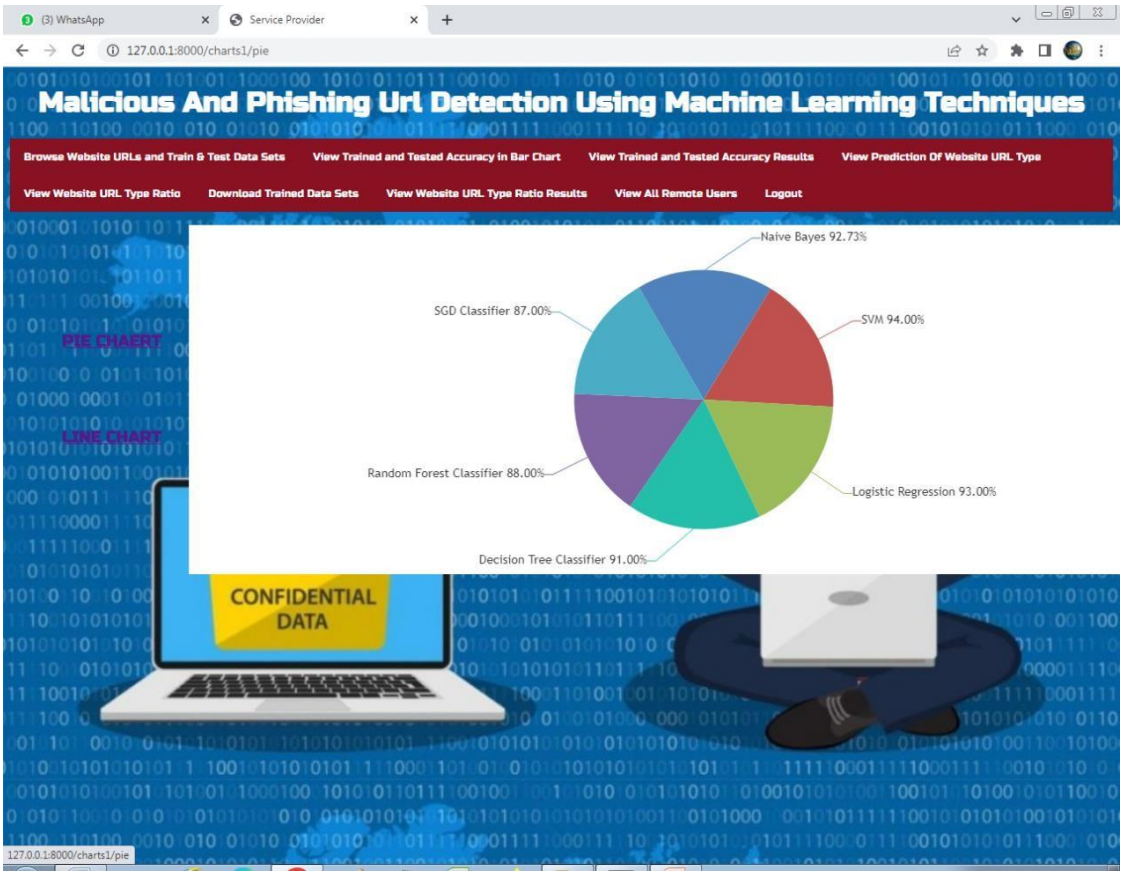
Screen 6: ML Accuracy



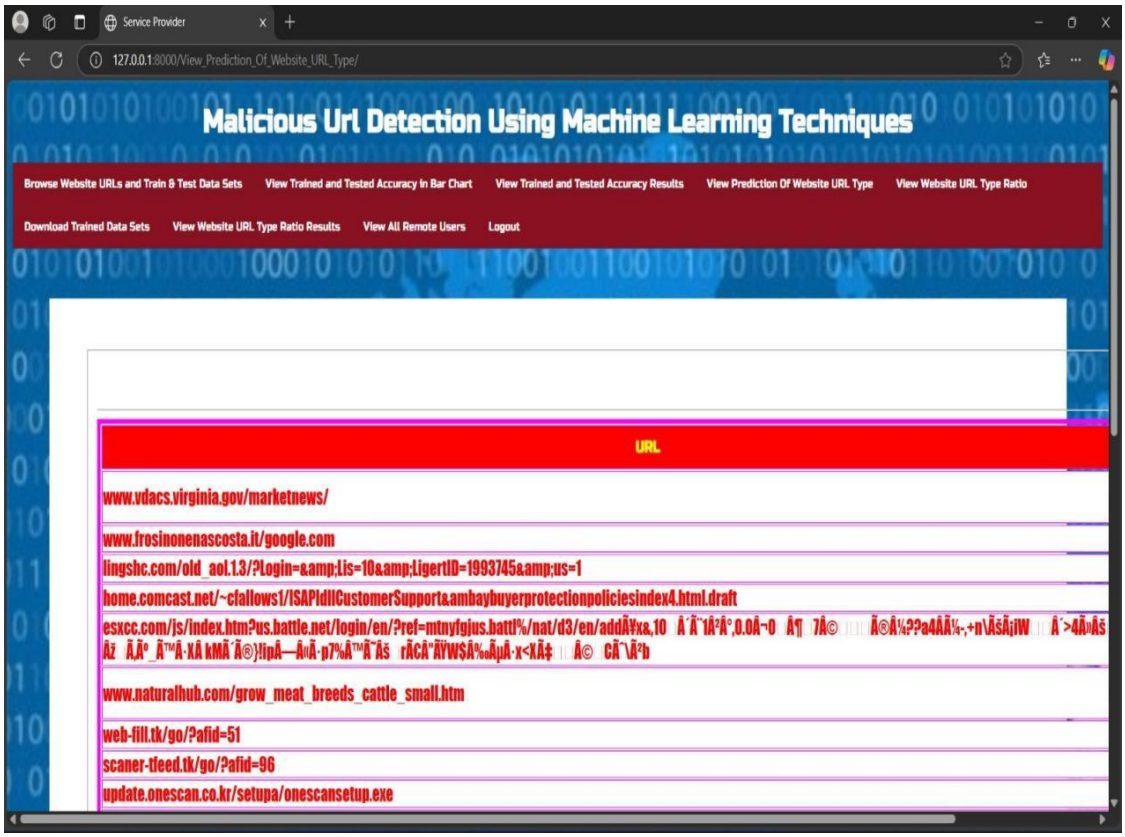
Screen 7: Graph Showing ML Accuracy



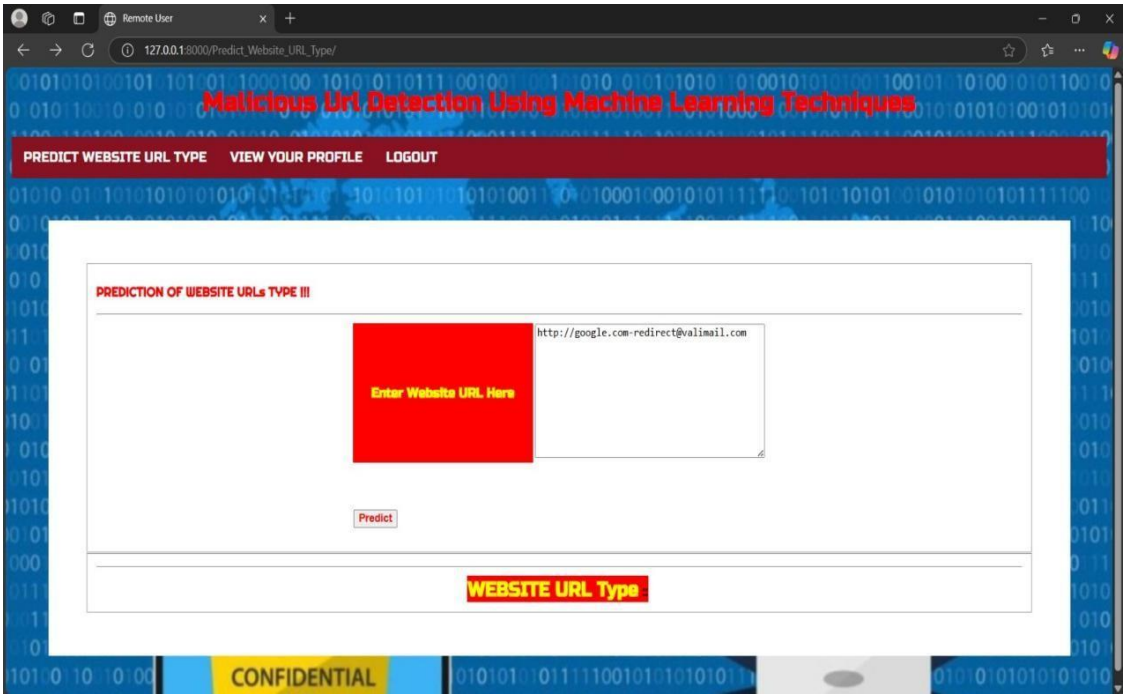
Screen 8: Line Graph Showing ML Accuracy



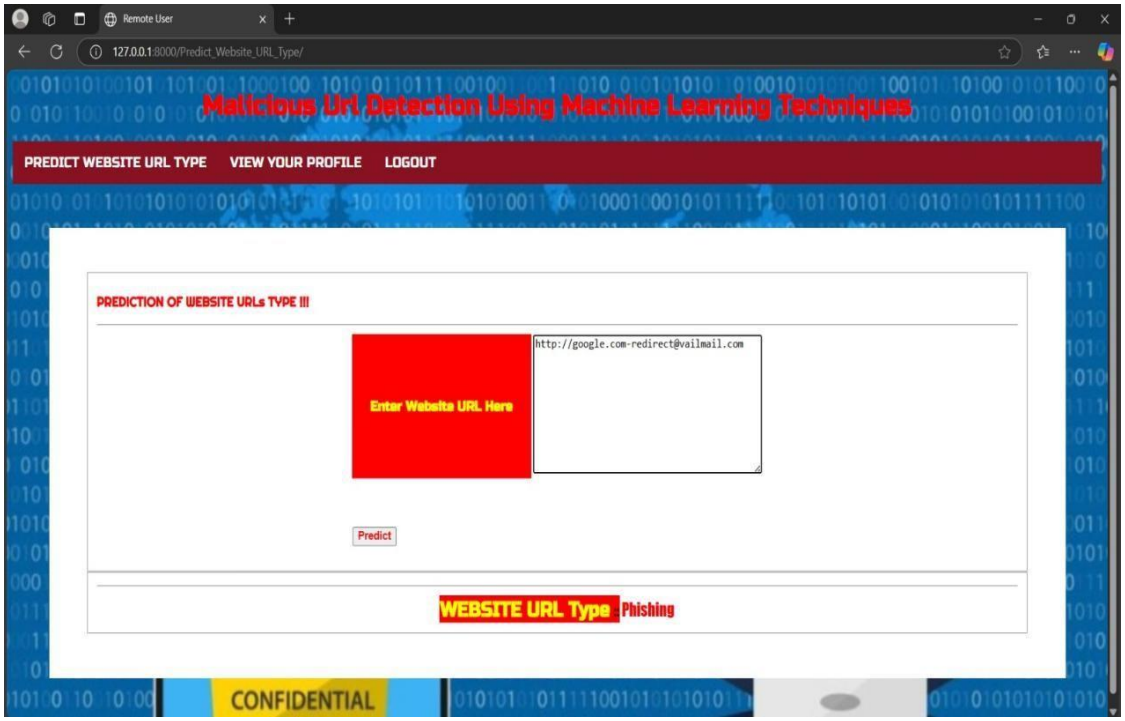
Screen 9: Pie Graph Showing ML Accuracy



Screen 10: List of Fares with Status



Screen 11: Prediction of Claims



Screen 12: Prediction Results

10. CONCLUSION AND FUTURE SCOPE

10.1 CONCLUSION

Phishing detection is now an area of great interest among the researchers due to its significance in protecting privacy and providing security. There are many methods that perform phishing detection by classification of websites using trained machine learning models. URL based analysis increases the speed of detection. Furthermore, by applying feature selection algorithms and dimensionality reduction techniques, we can reduce the number of features and remove irrelevant data. There are many machine learning algorithms that perform classification with good performance measures. In this paper, we have done a study of the process of phishing detection and the phishing detection schemes in the recent research literature. This will serve as a guide for new researchers to understand the process and to develop more accurate phishing detection systems.

10.2 FUTURE SCOPE

Further present a multiparty access control mechanism over the cipher text, which allows the data co-owners to append their access policies to the cipher text. Besides, we provide three policy aggregation strategies including full permit, owner priority and majority permit to solve the problem of privacy conflicts. In the future, we will enhance our scheme by supporting keyword search over the cipher text..

11. REFERENCES

1. Anti-phishing Working Group (APWG) Phishing Activity Trends Report 4th quarter 2020, https://docs.apwg.org/reports/apwg trends report q4 2020.pdf

2. FBI Internet Crime Report 2020, https://www.ic3.gov/Media/PDF/AnnualReport/2020 IC3Report.pdf

3. Verizon 2020 Data Breach Investigation Report, https://enterprise.verizon.com/resources/reports/2020-data- breachinvestigations-report.pdf

4. World Health Organization, Communicating for Health, Cyber Security, https://www.who.int/about/communications/cyber-security

5. Ye Cao, Weili Han, and Yueran Le, Anti-phishing based on automated individual white-list, Proceedings of the 4th ACM workshop on Digital identity management-DIM 08, pp. 51-60, 2008

6. M. Sharifi, and S. H. Siadati, A phishing sites blacklist generator, 2008 IEEE/ACS International Conference on Computer Systems and Applications, pp. 840-843, 2008

7. N. Abdelhamid, A. Ayesh, and F. Thabtah, Phishing detection based associative classification data mining, Expert Systems with Applications, vol. 41, no.13, pp. 5948-5959, 2014

8. L. Wenyin, G. Huang, L. Xiaoyue, Z. Min, and X. Deng, Detection of phishing webpages based on visual similarity, Special interest tracks and posters of the 14th international conference on World Wide Web-WWW 05, pp. 1060-1061, 2005

9. C. L. Tan, K. L. Chiew et al., Phishing website detection using url assisted brand name weighting system, 2014 International Symposium on Intelligent Signal Processing and Communication Systems(ISPACS), IEEE, pp. 054-059, 2014

11.1 REFERENCE BOOKS

- Python Crash Course 2nd Edition this is a basic-level book for beginners.
- Learning Python 5th Edition - this book is a practical learning book for basic to advanced level.
- Python Cookbook - this book is for advanced programmers interested in learning about modern Python development tools.
- Automating Boring Stuff With Python - In this book, you will learn to write programs in Python.
- Head First Python - this book covered the fundamentals of python.
- Think Python the basics of programming concepts and cover advanced topics like data structure and object-oriented design.